

Tools Needed:

Access to OpenSSH Client

- 1) [WSL \(recommended for Windows users\)](#)
- 2) Command Line / Terminal (Mac)
- 3) Linux ... (If you're using this, I don't have to tell you.)

Accessing the Jenkins Server

- 1) Create an SSH Tunnel from your machine to the Jenkins Server
 - a. Servers
 - i. osc-mgmt.eng.auburn.edu
 1. username: comp2710
 2. password: comp2710

Example:

```
ssh -L 8002:192.168.0.197:8080 comp2710@osc-mgmt.eng.auburn.edu
```

Command Breakdown:

-L forward a local port to a remote machine

Blue (8002) is the local port that you are forwarding traffic from (your laptop/desktop).

Orange (8080) is the port on the remote machine that the jenkins server is listening on.

192.168.0.197 is the remote machine that the Jenkins server is on.

You can optionally use a jump flag if you need to log in to multiple servers in order to get to osc-mgmt, such as if you logged in through the engineering gateway instead of using the vpn (gate.eng.auburn.edu).

-J "jump" through multiple machines to get to the final destination. Format is comma separated list with final destination separated with space. Ex: *server1,server2,server3 final_server*

- 2) Open any web browser and go to the remote machine part of the address in the tunnel you made, followed by the local port that you are forwarding. For this example, this would be **localhost:8002**

localhost:8002/login?from=%2F

Auburn Email Auburn Canvas AU Access vocareum Zybooks Lemongrass Gmail Rodrigo Gmail Other favorites



Sign in to Jenkins

Username

Password

☐ Keep me signed in

Sign in

[or register](#)

3) If this is your first time accessing the server, register and create an account.

localhost:8002/login?from=%2F

Auburn Email Auburn Canvas AU Access vocareum Zybooks Lemongrass Gmail Rodrigo Gmail Other favorites



Sign in to Jenkins

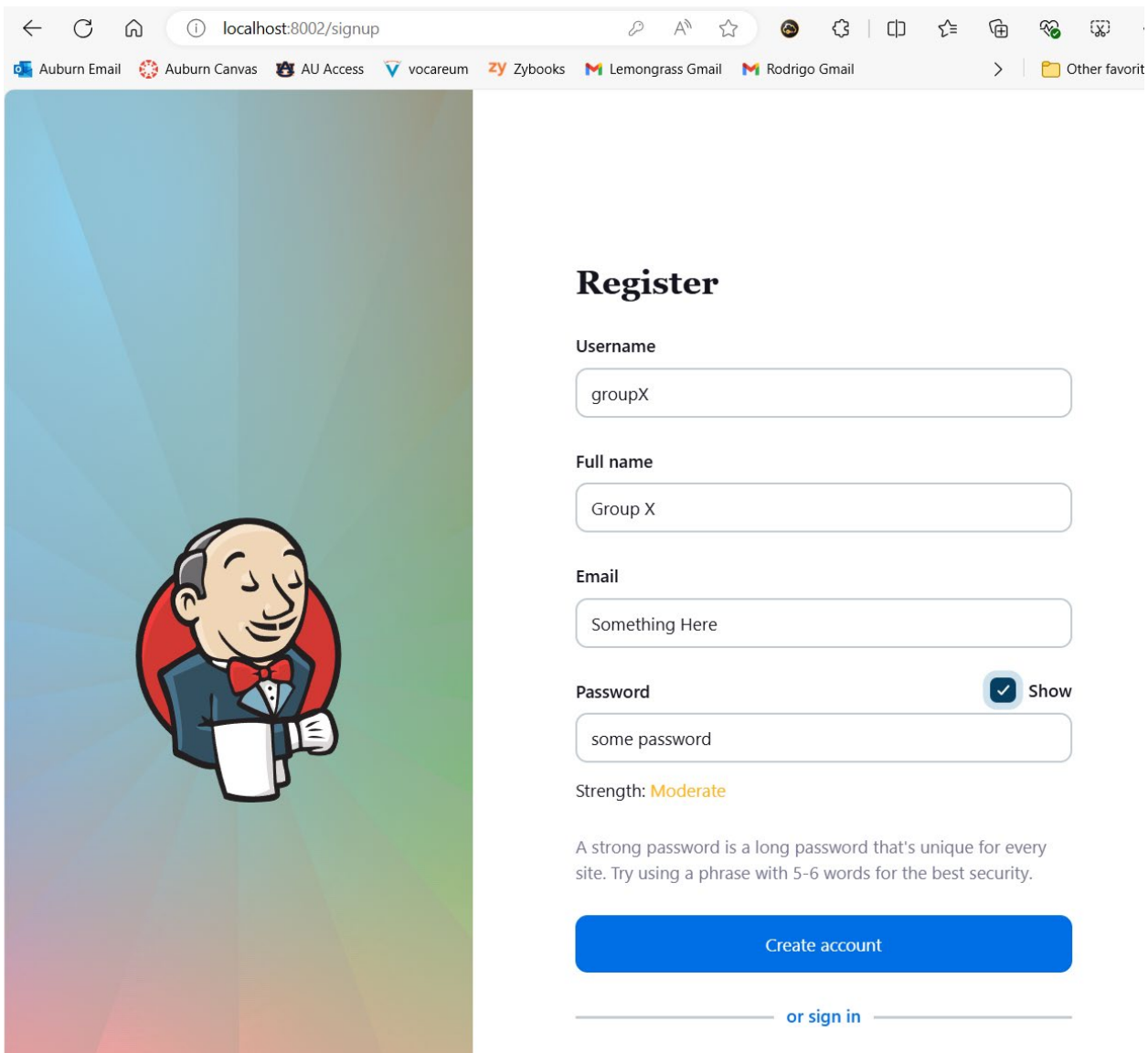
Username

Password

☐ Keep me signed in

Sign in

[or register](#)



localhost:8002/signup

Auburn Email Auburn Canvas AU Access vocareum Zy Zybooks Lemongrass Gmail Rodrigo Gmail Other favorit

Register

Username

groupX

Full name

Group X

Email

Something Here

Password ☒ Show

some password

Strength: **Moderate**

A strong password is a long password that's unique for every site. Try using a phrase with 5-6 words for the best security.

Create account

[or sign in](#)

- Be sure to use your auburn ID as your username.
- Be sure to use your username_hw1 as the job name for this first assignment.
 - Example Job Name: ras0054_hw1
- For full name, you can put your name.

Using Jenkins

The Jenkins server is a shared server. Please use responsibly and DO NOT modify or delete or run other people's jobs. You will all be using the same Jenkins server, meaning that you will have access to everyone's jobs.

Please Limit the number of jobs created to 1 job per assignment per group. For example, let's say there were 24 people in the class. For homework 1, each group will create a single job. That's 24 jobs. For the second assignment, each group will create another job. That would be a total of 48 jobs.

Naming Conventions:

In order to help the TA and I with grading, please name your jobs as follows:

username-HW Y Where Y is the assignment number. It doesn't matter if it's upper or lower case.

For example:

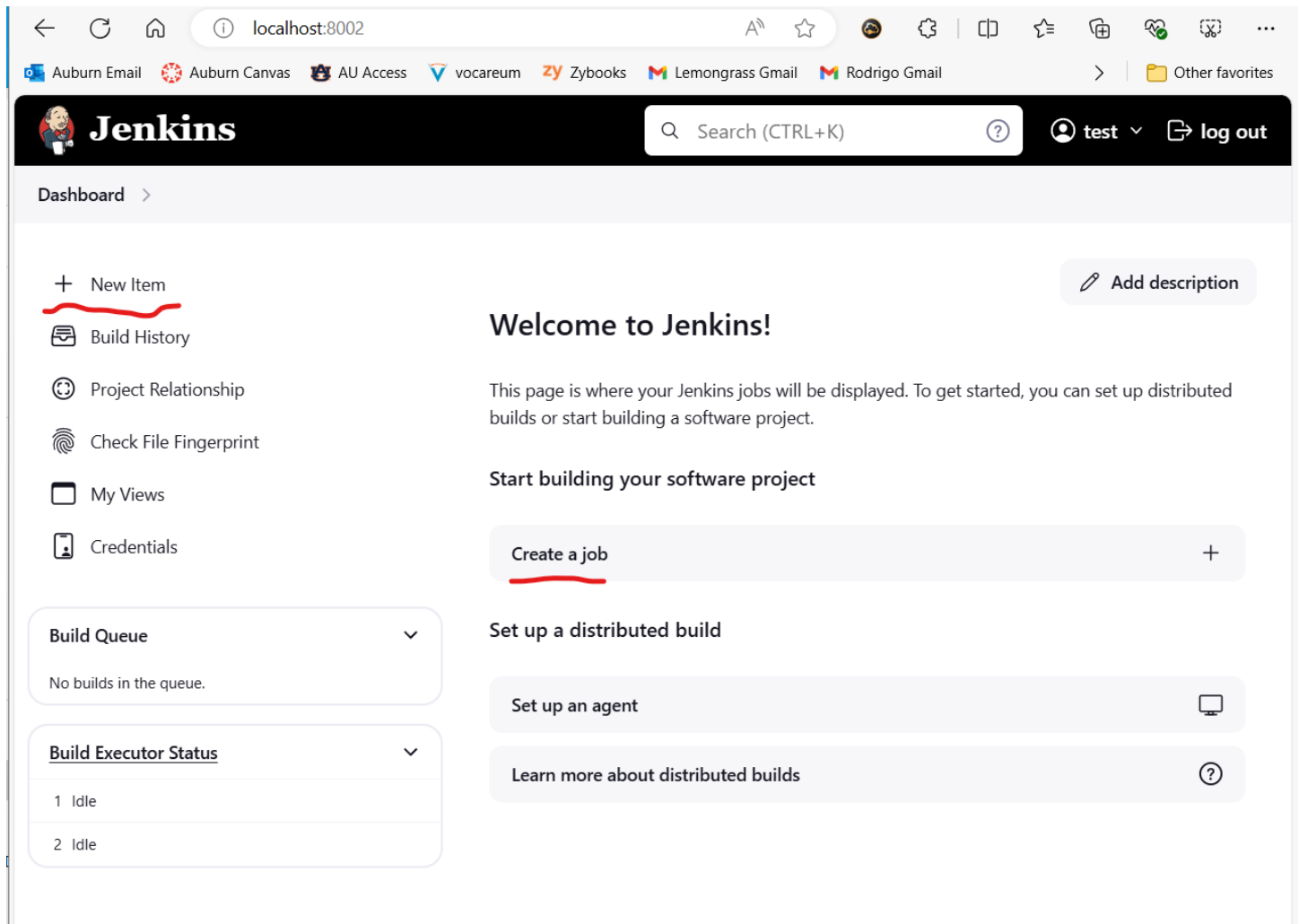
ras0054-HW1 or **ras0054-hw1** are both fine.

This will make it easier for the TA to grade your assignments as he/she can easily create a filter for all assignments using the assignment number as a label.

Creating a Jenkins Job

Jenkins is a tool that has the ability to automate tasks for you. For example, we are going to be using it to fetch code from a github repo, pull it locally, compile it, and run it. It can do other things (like launch docker containers, create vms, etc...

One way that you specify what kind of tasks you want it to do for you is by creating a job. A job is one or more tasks that you want to automate.



- 1) If there are no jobs on the server, your dashboard will look something like this. Select Create a job. You can also select “New Item”.

localhost:8002/view/all/newJob

Auburn Email Auburn Canvas AU Access vocareum Zybooks Lemongrass Gmail Rodrigo Gmail Other f

Jenkins

Search (CTRL+K) test log

Dashboard > All > New Item

New Item

Enter an item name

group1-hw1

Select an item type

-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
-  **Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.
-  **Organization Folder**
Creates a set of multibranch project subfolders by scanning for repositories.

OK

- 2) For now, select “Freestyle project”. Note that the “item name” is groupX-hwX. The item name is also the job name. Be sure to check “Discard old builds” to save disk space on the server.

 General

 Source Code Management

 Build Triggers

 Build Environment

 Build Steps

 Post-build Actions

Description

Plain text [Preview](#)

☒ Discard old builds 

Strategy

Log Rotation 

Days to keep builds

if not empty, build records are only kept up to this number of days

Max # of builds to keep

if not empty, only up to this number of build records are kept

1

Advanced 

- 3) Selecting ok will take you to a page where you can configure your job. For now, we want to configure several things.
- Our github repo (each assignment you'll be assigned a new github repo that you can use to upload your work for that specific assignment).

Dashboard > group1-hw1 > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

☐ None

☒ Git ?

Repositories ?

Repository URL ?

Please enter Git repository.

Credentials ?

- none -

+ Add

Advanced

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Add

Save

Apply

The screenshot shows a GitHub repository page for 'bowling-game-sprint-1-frames-test-team', which is a fork of 'AuburnCseRsardinas/bowling_frames'. The repository is private and has 1 branch and 0 tags. A 'Clone' dropdown menu is open, showing options for Local and Codespaces. Under the 'Local' tab, the 'Clone' button is selected, and the 'HTTPS' method is chosen. The URL 'https://github.com/AuburnCseRsardinas/bowling-' is displayed and highlighted with a red line. Below the URL, the text 'Clone using the web URL.' is visible. Other options in the dropdown include 'SSH', 'GitHub CLI', 'Open with GitHub Desktop', and 'Download ZIP'. The repository's file list includes '.github/workflows', 'headers', '.gitignore', 'README.md', 'bowling_frame.cpp', and 'bowling_frame.h'. The right sidebar shows the 'About' section with the description 'sprint 1 for bowling assignment' and the 'Releases' section with the text 'No releases published'.

- i. The repository url is the url you would use to clone the github repo.

Configure

General

Source Code Management

Build Triggers

Build Environment

Build Steps

Post-build Actions

The screenshot shows the 'Configure' dialog box for adding a repository. The 'Git' tab is selected. The 'Repository URL' field contains the URL 'https://github.com/AuburnCseRsardinas/bowling-game-sprint-1-frames-test-team.git'. The 'Credentials' dropdown menu is open, showing the selected credential 'test-ghc/*****'. A red line is drawn under the 'Add' button. The 'Advanced' dropdown menu is also visible. The 'Add Repository' button is at the bottom.

- b. Next you need to either select some credentials that will give you access to that repo (if you've already created them), or you need to create them. I have found that it is easiest to create an api key and use that to access all of your github repos. This will be covered in another section of this document.

Git ?

Repositories ?

Repository URL ?

https://github.com/AuburnCseRsardinas/bowling-game-sprint-1-frames-test-team.git

Credentials ?

test-ghc/*****

+ Add ^

Jenkins

Advanced v

Jenkins Credentials Provider: Jenkins

Add Credentials

Domain

Global credentials (unrestricted)

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

test_gch

☐ Treat username as secret ?

Password ?

Api Key

ID ?

Description ?

Cancel

Add

- c. Username should be your github username (only one of the group members needs to do this).
- d. Password should be the API key you created in github.

Configure

General

Source Code Management

Build Triggers

Build Environment

Build Steps

Post-build Actions

None

Git ?

Repositories ?

Repository URL ?

`https://github.com/AuburnCsseRsardinas/bowling-game-sprint-1-frames-test-team.git`

Failed to connect to repository : Command "git ls-remote -h -- https://github.com/AuburnCsseRsardinas/bowling-game-sprint-1-frames-test-team.git HEAD" returned status code 128:

stdout:

stderr: remote: Support for password authentication was removed on August 13, 2021. Please see <https://docs.github.com/get-started/getting-started-with-git/about-remote-repositories#cloning-with-https-urls> for information on currently recommended modes of authentication.

fatal: Authentication failed for 'https://github.com/AuburnCsseRsardinas/bowling-game-sprint-1-frames-test-team.git/'

Credentials ?

- current -

+ Add ▾

Cannot find any credentials with id test1

Advanced ▾

e. If you see this screen, (all the red text) then most likely your credentials aren't working correctly.

Configure

General

Source Code Management

Build Triggers

Build Environment

Build Steps

Post-build Actions

None

Git ?

Repositories ?

Repository URL ?

`https://github.com/AuburnCsseRsardinas/bowling-game-sprint-1-frames-test-team.git`

Credentials ?

test-ghc/*****

+ Add ▾

Advanced ▾

f. Note how the warning goes away once the correct credentials are selected.

Configure

- General
- Source Code Management
- Build Triggers**
- Build Environment
- Build Steps
- Post-build Actions

Add Branch

Repository browser ?
(Auto) ▼

Additional Behaviours
Add ▼

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☐ Build periodically ?

☐ GitHub hook trigger for GITScm polling ?

☒ Poll SCM ?

Schedule ?
H/60 * * * * *

Would last have run at Thursday, August 22, 2024, 1:11:30 AM Coordinated Universal Time; would next run at Thursday, August 22, 2024, 2:11:30 AM Coordinated Universal Time.

4) Next we want to configure what triggers a “build”. A build is an activation of our job.

- Since our Jenkins server doesn’t have a public URL we can’t configure it to work with Github hook Triggers. Ideally this is what you would want for a CI/CD type job. With this type of trigger, everytime some event happens in your github repo (like someone pushing new code to a specific branch), github would notify your Jenkins server and this would trigger some job to run.
- Since we can’t do this, we will have to settle for checking periodically. The schedule uses CRON notation (automated linux task scheduler) to determine how often to run. For now, just configure it to run hourly (you can use exactly what I did above). That means that each hour, go to check the github repo, and if there were any changes, trigger the build.
- Once you enter a schedule, you will see text below detailing exactly how often it is going to run by way of an example.
- You also want to select the option to abort the job if the log file gets too big. This usually happens (in this class) with infinite loops. **(I do this globally now, so you don’t have to worry about it.)**

Build Environment

- ☐ Delete workspace before build starts
- ☐ Use secret text(s) or file(s) ?
- ☒ Abort the build if its log file size is too big

Use job specific maximum log size instead global value (0 MB)

- ☒ Use job specific size

Job specific size ?

Job specific max log size in MB

- ☒ Fail the build ?

If checked, the build will be marked as failed rather than marked as aborted. (In any case, the build does not continue when the maximum file size is reached).

(from [Jenkins build log file size checker plugin](#))

- ☐ Add timestamps to the Console Output
- ☐ Inspect build log for published build scans
- ☐ Terminate a build if it's stuck
- ☐ With Ant ?

Build Steps

Add build step ^

Filter

- Execute Windows batch command
- Execute shell
- Invoke Ant
- Invoke Gradle script
- Invoke top-level Maven targets
- Run with timeout
- Set build status to "pending" on GitHub commit

- 5) Next, we want to configure what actions the job is going to take. For now, we will simply execute shell commands. For all of your homework assignments, you should create makefiles (covered later) that will compile your code for you. We will simply execute those files, then run the executable file that was created.

Build Steps

≡ **Execute shell** ?

Command

See [the list of available environment variables](#)

```
make
```

Advanced ▾

Add build step ^

Filter

Execute Windows batch command

Execute shell

Invoke Ant

Invoke Gradle script

Invoke top-level Maven targets

Run with timeout

Set build status to "pending" on GitHub commit

Configure

-  General
-  Source Code Management
-  Build Triggers
-  Build Environment
-  **Build Steps**
-  Post-build Actions

Build Steps

≡ **Execute shell** ?



Command

See [the list of available environment variables](#)

```
make
```

Advanced ▾

≡ **Execute shell** ?



Command

See [the list of available environment variables](#)

```
./a.out
```

Advanced ▾

Add build step ▾

Post-build Actions

Save

Apply

- 6) Finally, select the option to clean up the workspace when the build is complete. Again, this is to save disk space on the Jenkins server. (This is optional as I've allocated much more space this year for the Jenkins server.)

Post-build Actions

Delete workspace when build is done

Advanced ▾

Add post-build action ▾

Save

Apply

Record fingerprints of files to track usage

Git Publisher

E-mail Notification

Editable Email Notification

Set GitHub commit status (universal)

Set build status on GitHub commit [deprecated]

Delete workspace when build is done

Add post-build action ^

Save

Apply

- 7) When you're done, you can click save to save the job.

Dashboard > group1-hw1 >

Status

</> Changes

Workspace

Build Now

Configure

Delete Project

Git Polling Log

Rename

group1-hw1

Permalinks

Add description

Build History

trend ▾

No builds

Atom feed for all Atom feed for failures

Once you save, it should take you to this screen where you can see information about the job, among other things. Note that you can manually trigger the build (job) and modify it (configure) among other things.

Jenkins

Search (CTRL+K) test ▾ log out

Dashboard > group1-hw1 >

Status

</> Changes

Workspace

Build Now

Configure

Delete Project

Git Polling Log

Rename

group1-hw1

Permalinks

Add description

Build History

trend ▾

Filter...

#1

Aug 22, 2024, 2:10 AM

Atom feed for all Atom feed for failures

Once the build has run at least once, you will see a build history (each time the build was triggered). If no changes are made, it should not trigger again. If you make changes to your github repo, then whenever it polls your github repo, it should trigger a new build.

 **Jenkins**

Search (CTRL+K) ? test log out

Dashboard > group1-hw1 > #1

Status

</> Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

✓ #1 (Aug 22, 2024, 2:10:07 AM)

Add description

Keep this build forever

Started by user test

Started 2 min 3 sec ago
Took 0.96 sec

This run spent:

- 46 ms waiting;
- 0.96 sec build duration;
- 1 sec total from scheduled to completion.

Revision: 0cb9ca75b4bcca9e115f25bb3e46b9712aab5fc6

Repository: <https://github.com/AuburnCsseRsardinas/bowling-game-sprint-1-frames-test-team.git>

- origin/main

</> No changes.

Selecting a build number will take you to a page that will show you information about that instance of the job running.

 **Jenkins**

Search (CTRL+K) ? test log out

Dashboard > group1-hw1 > #1 > Console Output

Status

</> Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

✓ Console Output

Download

Copy

View as plain text

```
Started by user test
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/group1-hw1
The recommended git tool is: NONE
using credential api-token
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/group1-hw1/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/AuburnCsseRsardinas/bowling-game-sprint-1-frames-test-team.git # timeout=10
Fetching upstream changes from https://github.com/AuburnCsseRsardinas/bowling-game-sprint-1-frames-test-team.git
> git --version # timeout=10
> git --version # 'git version 2.34.1'
using GIT_ASKPASS to set credentials
> git fetch --tags --force --progress -- https://github.com/AuburnCsseRsardinas/bowling-game-sprint-1-frames-test-team.git +refs/heads/*:refs/remotes/origin/* # timeout=10
Seen branch in repository origin/main
Seen 1 remote branch
> git show-ref --tags -d # timeout=10
Checking out Revision 0cb9ca75b4bcca9e115f25bb3e46b9712aab5fc6 (origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 0cb9ca75b4bcca9e115f25bb3e46b9712aab5fc6 # timeout=10
Commit message: "Delete Jenkinsfile"
First time build. Skipping changelog.
[group1-hw1] $ /bin/sh -xe /tmp/jenkins14939086462858623608.sh
+ make
g++ -std=c++11 bowling_frame.o main.o
[group1-hw1] $ /bin/sh -xe /tmp/jenkins9471278698738169999.sh
+ ./a.out
0
Score 1: 8
Score 2: 2
Next Frame Score 1: 7
Finished: SUCCESS
```

The tab we really care about will be the console output tab. Your submitted assignments should always produce output. When your executable is run, we can see that output here. Your assignments will be graded based on the output here.

The screenshot shows the Jenkins Dashboard. At the top, there's a header with the Jenkins logo, a search bar, and user information. Below the header, there's a sidebar with navigation links: New Item, Build History, Project Relationship, Check File Fingerprint, My Views, and Credentials. The main area displays a table of jobs. The first job is 'group1-hw1', which is in a successful state (green checkmark). The table columns are: S (Status), W (Workspace), Name, Last Success, Last Failure, and Last Duration. Below the table, there are two dropdown menus: 'Build Queue' and 'Build Executor Status'. The 'Build Queue' dropdown shows 'No builds in the queue.' The 'Build Executor Status' dropdown shows '1 Idle' and '2 Idle'.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	group1-hw1	5 min 22 sec #1	N/A	0.96 sec

If you go back to your dashboard, your default view will show you all jobs in the Jenkins server. This is one reason we want to have some kind of naming convention for jobs, to ensure that you ONLY work with or modify your jobs or even the job for the specific assignment. From here, you can select a job, rerun it, make modifications, etc...

This screenshot is similar to the previous one, but with a context menu open for the job 'group1-hw1'. The menu is triggered by clicking on the job name. The menu options are: Changes, Workspace, Build Now, Configure, Delete Project, Git Polling Log, and Rename. Red annotations highlight the 'group1-hw1' job name and the 'Delete Project' option in the menu.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	group1-hw1	6 min 44 sec #1	N/A	0.96 sec

- Changes
- Workspace
- Build Now
- Configure
- Delete Project
- Git Polling Log
- Rename

Please be careful NOT to delete anyone else's jobs.